PROBLEM 2 · SOLUTION

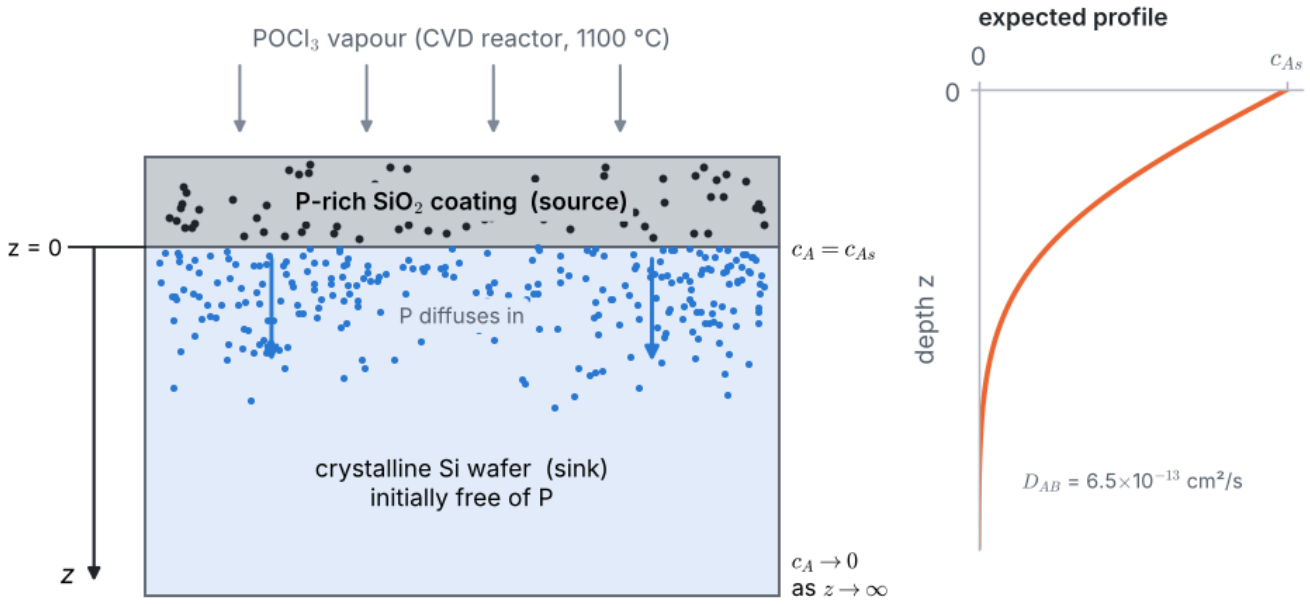
Phosphorus diffusion into silicon

Step-by-step solution with the Finite Difference Method

Plan. Problem 1 was steady; this one is **unsteady**: the concentration depends on depth *and* time, so we march forward in time. We follow the 7 pillars of CFD: geometry → grid → physics → numerical scheme → solution setup → compute → post-processing. All code is plain Python (lists and loops only; `math` for the exact solution) and is the code of the notebook `03_fdm_solver.ipynb`.

Step 1 · Geometry and data

The P-rich SiO_2 coating on top of the wafer keeps the surface concentration at $c_{As} = 2.5 \times 10^{20}$ atoms/cm³. Phosphorus diffuses down into the silicon, which starts free of P. Because D_{AB} is tiny, the phosphorus penetrates only micrometres into a wafer hundreds of micrometres thick, so the wafer is treated as **semi-infinite** ($0 \le z < \infty$).



The doping process and the expected shape of the concentration profile.

```
# ---- process data ------------------------------------------------------
T_process = 1100.0    # process temperature                  [°C]
c_As = 2.5e20         # surface concentration of P           [atoms/cm^3]
c_A0 = 0.0            # initial concentration of P in the wafer [atoms/cm^3]
D_cm2 = 6.5e-13       # diffusivity of P in Si at 1100 °C    [cm^2/s]
t_end = 3600.0        # doping time: 1 hour                  [s]
target = 0.01         # film edge: c_A = 1 % of the surface value
```

Convenient units. Micrometres and seconds give friendly numbers: $1\ \mathrm{cm}^2 = 10^8\ \mu\mathrm{m}^2$, so $D = 6.5 \times 10^{-5}$ µm²/s. The **diffusion length** $\sqrt{Dt}$ tells roughly how far atoms travel in a time t:

```
D = D_cm2 * 1e8                    # diffusivity in µm^2/s

diffusion_length = math.sqrt(D * t_end)
print(f"D = {D:.2e} µm^2/s")
print(f"diffusion length after 1 h:  sqrt(D t) = {diffusion_length:.3f} µm")
```

```
D = 6.50e-05 µm^2/s
diffusion length after 1 h:  sqrt(D t) = 0.484 µm
```

Step 2 · The grid in space and time

Space. A computer grid must end somewhere, so the semi-infinite wafer is cut at $z_{max} = 5$ µm, about ten diffusion lengths. The phosphorus cannot get that far in one hour, so replacing "$c_A \to 0$ as $z \to \infty$" by "$c_A = 0$ at z_{max}" changes nothing (this is checked in Step 7). Nodes: $z_i = i\,\Delta z$, $i = 0, \ldots, N$.

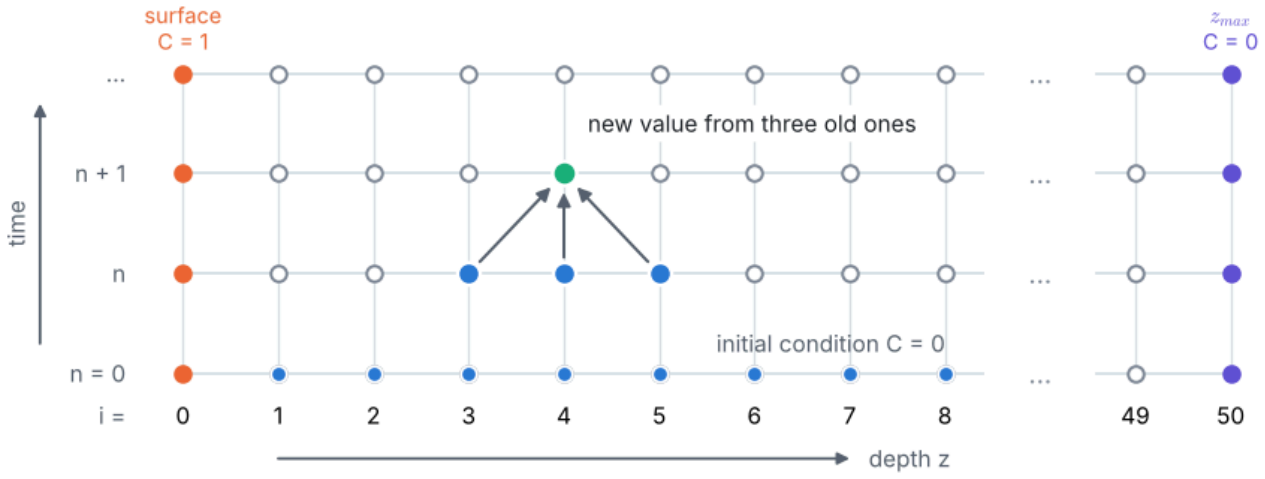
Time. Time is cut into steps Δt; the unknowns are C_i^n, the concentration at node i and time $t^n = n\,\Delta t$.

```
z_max = 5.0                # depth where we cut the semi-infinite wafer [µm]
dz = 0.1                   # grid spacing                               [µm]
N = round(z_max / dz)      # number of intervals

z = []                     # depth of every node [µm]
for i in range(N + 1):
    z.append(i * dz)

print(f"N = {N} intervals, dz = {dz} µm, {len(z)} nodes from z = 0 to z = {z[-1]:.1f} µm")
print(f"z_max = {z_max / diffusion_length:.1f} diffusion lengths")
```

```
N = 50 intervals, dz = 0.1 µm, 51 nodes from z = 0 to z = 5.0 µm
z_max = 10.3 diffusion lengths
```



The space–time grid. Bottom row: initial condition. Left column: surface (known). Right column: the cut at z_{max} (zero). The explicit scheme computes each new value from three values of the previous time level.

Step 3 · Physics: the diffusion equation

Mass balance for phosphorus on a slice between z and $z + dz$ (area A):

$$\frac{\partial c_A}{\partial t}\,A\,dz = J_z A - J_{z+dz} A = -\frac{\partial J_z}{\partial z}\,A\,dz.$$

With **Fick's first law**, $J_z = -D_{AB}\,\partial c_A/\partial z$, this gives **Fick's second law**:

$$\frac{\partial c_A}{\partial t} = D_{AB}\,\frac{\partial^2 c_A}{\partial z^2}.$$

	condition	meaning
initial condition	$c_A(z,0) = c_{A0} = 0$	the wafer starts free of P
surface ($z = 0$)	$c_A(0,t) = c_{As}$	the coating keeps the surface concentration constant
deep inside ($z \to \infty$)	$c_A(\infty,t) = 0$	far away the wafer is untouched

Dimensionless concentration. With $C = c_A/c_{As}$ the equation keeps its form, $\partial C/\partial t = D\,\partial^2 C/\partial z^2$, with $C(z,0) = 0$, $C(0,t) = 1$, $C(\infty,t) = 0$. The film edge is where $C = 0.01$.

Exact solution. For the semi-infinite medium,

$$\frac{c_{As} - c_A}{c_{As} - c_{A0}} = \operatorname{erf}\!\left(\frac{z}{2\sqrt{D_{AB}\,t}}\right) \qquad \Longleftrightarrow \qquad C = 1 - \operatorname{erf}\!\left(\frac{z}{2\sqrt{Dt}}\right).$$

At the film edge, $\operatorname{erf}(\eta) = 0.99$ with $\eta = z/(2\sqrt{Dt})$, so $\eta \approx 1.82$ (erf tables). The code finds η precisely by **bisection** (halving an interval that contains the answer):

```python
def C_exact(zz, t):
    """Exact dimensionless concentration c_A/c_As at depth zz [µm] and time t [s]."""
    return 1.0 - math.erf(zz / (2.0 * math.sqrt(D * t)))


# solve erf(eta) = 1 - target = 0.99 by bisection
low, high = 0.0, 5.0
for _ in range(60):
    eta = 0.5 * (low + high)
    if math.erf(eta) < 1.0 - target:
        low = eta          # erf still too small: the answer is to the right
    else:
        high = eta         # erf too large: the answer is to the left


def depth_exact(t):
    """Exact depth [µm] where c_A = 1 % of c_As after a time t [s]."""
    return 2.0 * eta * math.sqrt(D * t)


print(f"erf(eta) = 0.99  ->  eta = {eta:.4f}")
print(f"exact film depth after 1 h:  z = 2 eta sqrt(D t) = {depth_exact(t_end):.4f} µm")
```

```
erf(eta) = 0.99  ->  eta = 1.8214
exact film depth after 1 h:  z = 2 eta sqrt(D t) = 1.7621 µm
```

Step 4 · Numerical scheme: explicit FTCS

At node i and time level n:

$$\left.\frac{\partial C}{\partial t}\right|_i^n \approx \frac{C_i^{n+1} - C_i^n}{\Delta t} \;\;(\text{forward}, \mathcal{O}(\Delta t)), \qquad \left.\frac{\partial^2 C}{\partial z^2}\right|_i^n \approx \frac{C_{i-1}^n - 2C_i^n + C_{i+1}^n}{\Delta z^2} \;\;(\text{central}, \mathcal{O}(\Delta z^2)).$$

Substituting into the diffusion equation and solving for the only unknown:

$$C_i^{n+1} = C_i^n + r\left(C_{i-1}^n - 2C_i^n + C_{i+1}^n\right), \qquad r = \frac{D\,\Delta t}{\Delta z^2}.$$

The scheme is **explicit**: every new value comes directly from three known old values, so there is no system of equations to solve. The boundary values are imposed at every step: $C_0^{n+1} = 1$ and $C_N^{n+1} = 0$.

```
def ftcs_step(C_old, r):
    """Advance the concentration one time step with the explicit FTCS scheme."""
    n_nodes = len(C_old)
    C_new = [0.0] * n_nodes

    C_new[0] = 1.0                # surface: c_A = c_As at all times
    for i in range(1, n_nodes - 1):
        C_new[i] = C_old[i] + r * (C_old[i - 1] - 2.0 * C_old[i] + C_old[i + 1])
    C_new[n_nodes - 1] = 0.0      # deep inside: no phosphorus

    return C_new
```

> The new values go into a **new list**. Overwriting the old list during the loop would make node i use the already-updated value of node $i - 1$, which is a different scheme.

Step 5 · Solution setup: a stable time step

Rewriting the update as

$$C_i^{n+1} = r\,C_{i-1}^{n} + (1 - 2r)\,C_i^{n} + r\,C_{i+1}^{n}$$

shows that the new value is a **weighted average** of three old values. While all weights are positive, the new value stays between its neighbours: no overshoot, no wiggles. The middle weight is positive only if

$$r = \frac{D\,\Delta t}{\Delta z^2} \le \frac{1}{2} \qquad \Longleftrightarrow \qquad \Delta t \le \frac{\Delta z^2}{2D}.$$

This is the **stability condition** of FTCS. We choose $\Delta t = 36$ s, well inside the limit and exactly 100 steps per hour:

```
dt_max = dz**2 / (2.0 * D)        # largest stable time step [s]
dt = 36.0                         # our choice [s]
r = D * dt / dz**2                # must be <= 0.5
n_steps = round(t_end / dt)

print(f"stability limit: dt <= {dt_max:.1f} s")
print(f"chosen: dt = {dt:.0f} s  ->  r = {r:.3f}  (stable, since r <= 0.5)")
print(f"number of time steps for 1 h: {n_steps}")
```

```
stability limit: dt <= 76.9 s
chosen: dt = 36 s  ->  r = 0.234  (stable, since r <= 0.5)
number of time steps for 1 h: 100
```

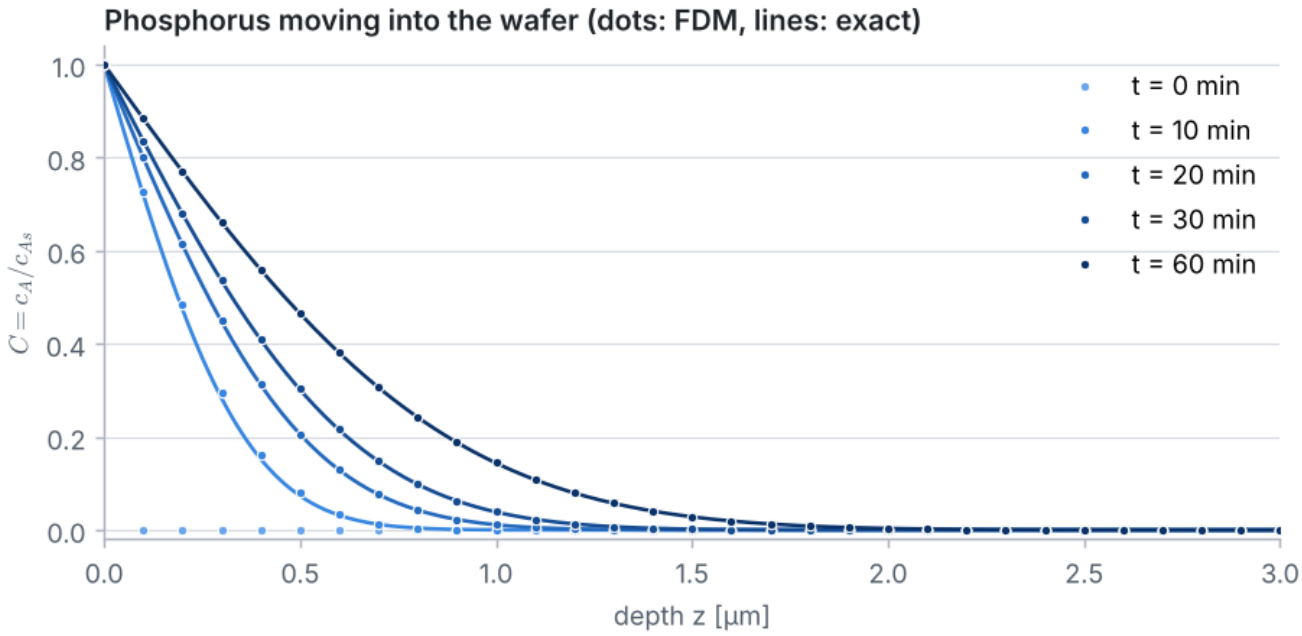
Step 6 · Compute: march through one hour

```
C = [0.0] * (N + 1)        # initial condition: no phosphorus in the wafer ...
C[0] = 1.0                 # ... except at the surface, held at c_As
history = [C[:]]           # history[n] = concentration profile at time level n

for n in range(1, n_steps + 1):
    C = ftcs_step(C, r)
    history.append(C)

print(f"done: {n_steps} steps, t = {n_steps * dt / 60:.0f} min")
print(" z [µm]   c_A/c_As (FDM)   exact")
for zz in (0.0, 0.5, 1.0, 1.5, 2.0, 2.5):
    i = round(zz / dz)
    print(f"{zz:6.1f}    {history[-1][i]:.6f}       {C_exact(zz, t_end):.6f}")
```

```
done: 100 steps, t = 60 min
 z [µm]   c_A/c_As (FDM)   exact
   0.0    1.000000       1.000000
   0.5    0.466040       0.464851
   1.0    0.144645       0.143806
   1.5    0.028512       0.028333
   2.0    0.003454       0.003461
   2.5    0.000251       0.000258
```



Concentration profiles after 10, 20, 30 and 60 minutes: FDM (dots) and exact solution (lines).

Step 7 · Post-processing and verification

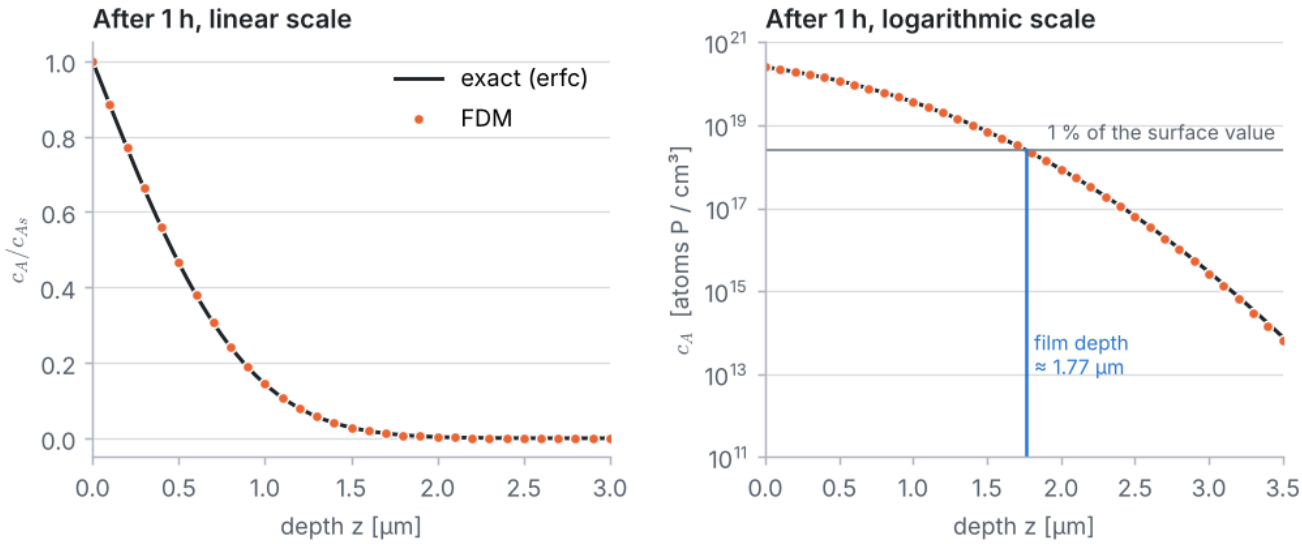
Film depth. The 1 % level rarely falls exactly on a node, so we interpolate linearly between the two nodes around it:

$$z_{film} = z_i + \frac{C_i - 0.01}{C_i - C_{i+1}}\,\Delta z, \qquad C_i \geq 0.01 > C_{i+1}.$$

```python
def film_depth(z, C, level):
    """Depth where the profile C first drops below 'level' (linear interpolation)."""
    for i in range(len(C) - 1):
        if C[i] >= level > C[i + 1]:
            return z[i] + (C[i] - level) / (C[i] - C[i + 1]) * (z[i + 1] - z[i])
    return 0.0                      # the level has not been reached anywhere yet


depth_fdm = film_depth(z, history[-1], target)
difference = 100 * (depth_fdm - depth_exact(t_end)) / depth_exact(t_end)
print(f"film depth after 1 h:  FDM {depth_fdm:.4f} µm    exact {depth_exact(t_end):.4f} µm"
      f"    ({difference:+.2f} %)")
```

```
film depth after 1 h:  FDM 1.7673 µm    exact 1.7621 µm    (+0.29 %)
```



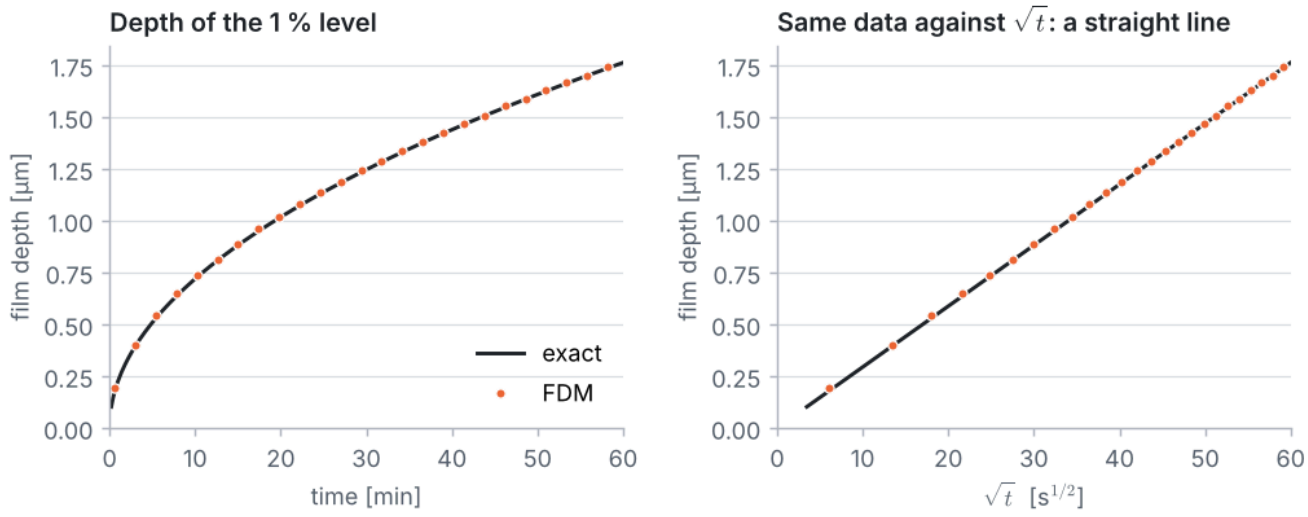
The profile after 1 h on a linear scale (left) and on a logarithmic scale in atoms/cm³ (right), where the 1 % level and the film depth can be read directly.

Growth of the film with time.

```python
times, depths = [], []
for n in range(1, n_steps + 1):
    times.append(n * dt)
    depths.append(film_depth(z, history[n], target))

for minutes in (15, 30, 60):
    n = round(minutes * 60 / dt)
    print(f"t = {minutes:2d} min:  FDM {depths[n - 1]:.3f} µm   exact {depth_exact(n * dt):.3f} µm")
```

```
t = 15 min:  FDM 0.887 µm   exact 0.881 µm
t = 30 min:  FDM 1.254 µm   exact 1.246 µm
t = 60 min:  FDM 1.767 µm   exact 1.762 µm
```



The film depth grows like $\sqrt{t}$: plotted against $\sqrt{t}$ it is a straight line. A film twice as deep takes four times as long.

Was z_{max} deep enough? The concentration near the cut after one hour:

```
for i in (N - 20, N - 10, N - 1):
    print(f"z = {z[i]:.1f} µm:  c_A/c_As = {history[-1][i]:.1e}")
```

```
z = 3.0 µm:  c_A/c_As = 1.1e-05
z = 4.0 µm:  c_A/c_As = 3.5e-09
z = 4.9 µm:  c_A/c_As = 2.9e-13
```

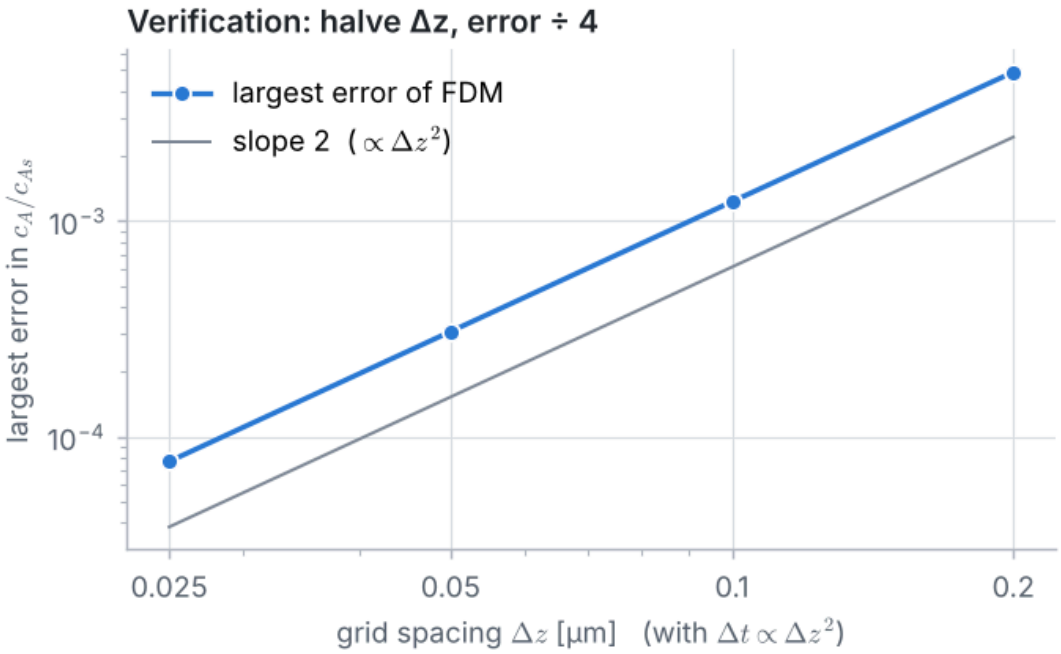
Next to the cut, the concentration is about 10^{-13} of the surface value: the phosphorus never felt the cut, so the finite grid represents the semi-infinite wafer.

Grid refinement. FTCS has error $\mathcal{O}(\Delta t) + \mathcal{O}(\Delta z^2)$. Halving Δz at fixed r divides Δt by 4, so both error terms should fall by a factor of 4:

```
def solve(dz, r):
    """Run the whole 1 h calculation on a grid with spacing dz at a fixed r."""
    N = round(z_max / dz)
    dt = r * dz**2 / D
    n_steps = round(t_end / dt)
    dt = t_end / n_steps                # land exactly on t = 1 h
    C = [0.0] * (N + 1)
    C[0] = 1.0
    for n in range(n_steps):
        C = ftcs_step(C, D * dt / dz**2)
    return [i * dz for i in range(N + 1)], C, n_steps


dzs, errors = [], []
print(" dz [µm]   steps   largest error   ratio   film depth [µm]")
for dz_test in (0.2, 0.1, 0.05, 0.025):
    z_t, C_t, steps = solve(dz_test, r)
    error = max(abs(C_t[i] - C_exact(z_t[i], t_end)) for i in range(len(z_t)))
    ratio = f"{errors[-1] / error:5.2f}" if errors else "     "
    dzs.append(dz_test)
    errors.append(error)
    print(f"{dz_test:7.3f}   {steps:5d}   {error:.2e}        {ratio}    {film_depth(z_t, C_t,
target):.4f}")
```

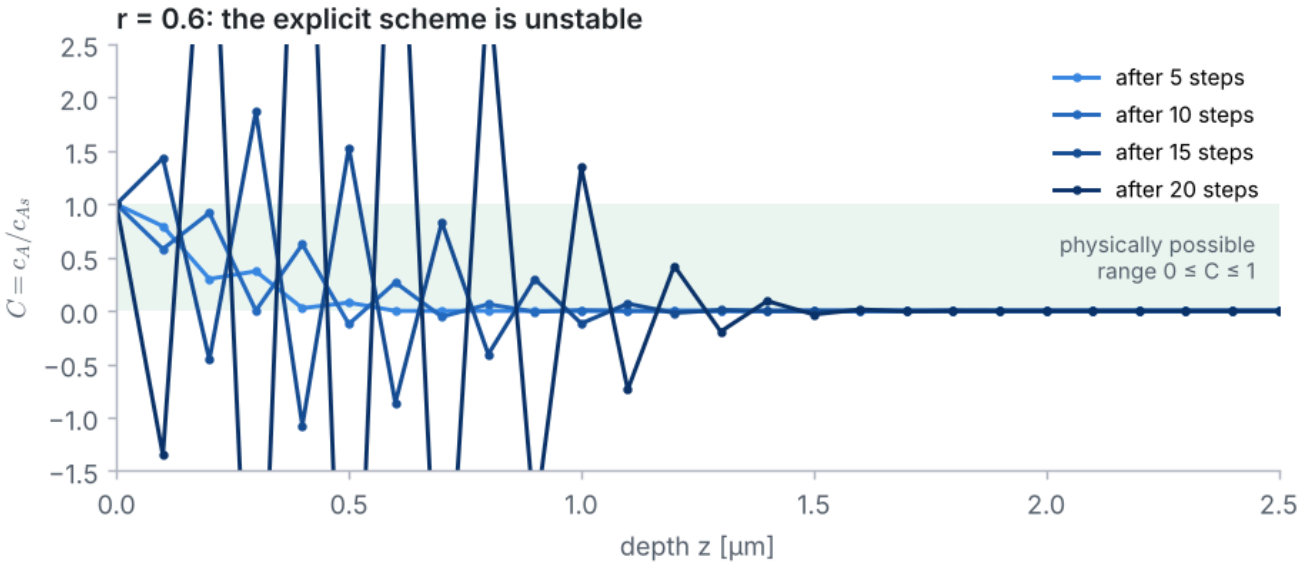
```
 dz [µm]   steps   largest error   ratio   film depth [µm]
  0.200      25   4.90e-03                  1.7750
  0.100     100   1.23e-03         3.98     1.7673
  0.050     400   3.08e-04         4.00     1.7632
  0.025    1600   7.69e-05         4.00     1.7625
```



Every halving of Δz (with $\Delta t \propto \Delta z^2$) divides the error by 4.

Breaking the stability limit. With $r = 0.6$ the middle weight $1 - 2r = -0.2$ is negative, and every step amplifies the differences between neighbours instead of smoothing them:

```python
r_bad = 0.6
C_bad = [0.0] * (N + 1)
C_bad[0] = 1.0
unstable_snapshots = {}
for n in range(1, 21):
    C_bad = ftcs_step(C_bad, r_bad)
    if n in (5, 10, 15, 20):
        unstable_snapshots[n] = C_bad[:]
```



With $r = 0.6$ a saw-tooth grows at every step; concentrations become negative or larger than the surface value, which is impossible.

Result. After 1 h, the FDM gives a film depth (1 % level, $c_A = 2.5 \times 10^{18}$ atoms/cm³) of **1.767 µm**, against **1.762 µm** from the exact solution, a difference of 0.3 %. The profile falls from 2.5×10^{20} atoms/cm³ at the surface to practically zero below about 3 µm, and the film depth grows like $\sqrt{t}$.

Key ideas

- Unsteady diffusion is governed by Fick's second law, with one initial and two boundary conditions.
- A semi-infinite domain is cut at a depth of many diffusion lengths, and the cut is checked afterwards.
- The explicit FTCS scheme is just an update formula, but it is stable only for $r = D\,\Delta t/\Delta z^2 \le 1/2$.
- Quantities of interest between nodes are found by interpolation.
- Refining the grid at fixed r verifies the code: the error falls by 4 per halving of Δz.